

SYSTEM DESIGN INTERVIEW

Design a Fitness Tracking System

Strava / Fitbit / Apple Health backend — wearables, time-series, social

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Sync Model — Device → Phone → Cloud
- 06 · Time-Series Storage Choice
- 07 · Aggregations & Daily Summary
- 08 · Workouts — GPX Tracks & HR Series
- 09 · Real-Time Leaderboards & Challenges
- 10 · Social Feed & Activity Posts

- [11](#) · HealthKit & Google Fit Integration
- [12](#) · Privacy — GPS Zones & Sharing Controls
- [13](#) · Failure Modes & Recovery
- [14](#) · Design Trade-offs
- [15](#) · Interview Playbook

01

Problem Statement

What we are building

Design the backend for a wearable fitness platform similar to **Strava**, **Fitbit**, or the **Apple Health** backend. The system ingests high-frequency sensor streams (steps, heart rate, GPS), aggregates them into daily / weekly / monthly summaries, surfaces insights and challenges, supports a social activity feed, and integrates two-way with Apple HealthKit and Google Fit.

Clarifying Questions

Question	Answer
How many users?	~500M registered, ~50M DAU sync
What sensors?	HR every 5s, steps every 1s, GPS only during workouts
Sync model?	Device → phone (BLE) → cloud (batched HTTP)
Retention?	30d hot, 1y warm, lifetime cold (S3/Glacier)
Social?	Friend graph, activity feed, kudos/comments, challenges
Integrations?	HealthKit and Google Fit two-way sync
Privacy?	Per-activity (private/friends/public) + GPS privacy zones
HIPAA?	HIPAA-adjacent; only triggered if integrated with a provider

Why this is interesting

- **Bursty, not steady-state ingest** — devices sync when phone gets connectivity, not in real time
- **Time-series at scale** — billions of HR samples per day with strict retention tiers
- **Two-write surface** — same data point can arrive from watch, phone sensor, and HealthKit
- **Sensitive geo data** — GPS tracks reveal home/work; privacy zones are mandatory
- **Social + health** — activity feed mechanics with auto-generated content (no user post)

02

Requirements

Functional and non-functional

Functional Requirements

Capability	Details
Sensor ingest	Accept batched samples (HR, steps, GPS, sleep, calories) from phone
Workout records	Start/stop a workout; persist GPS track + HR series + summary
Daily summary	Steps, active minutes, calories, distance, sleep, resting HR
Aggregations	Per-day at ingest; per-week and per-month rolled up nightly
Social feed	Friends, follow graph, activity feed, kudos, comments
Challenges	Weekly/monthly leaderboards (steps, distance) with prize tiers
Integrations	Two-way sync with Apple HealthKit and Google Fit
Privacy zones	Suppress GPS within radius of user-defined locations
Insights	Trend deltas, training load, personal records, anomaly nudges

Non-Functional Requirements

Requirement	Target
Availability	99.95% on read paths; 99.9% on ingest (offline-tolerant)
Ingest latency	<5 s p99 from phone POST to TSDB visibility
Read latency	<200 ms p99 for daily summary; <500 ms for workout map
Durability	11 nines on cold tier; 0 sample loss after phone ack
Throughput	Peak ingest ~25 GB/s aggregate at commute hours
Privacy	GPS encrypted at rest; per-user privacy zones enforced server-side
Eventual consistency	Feed/leaderboard may lag <30 s; summaries by next sync

03

Capacity Estimation

How big does this get

User and traffic

- Registered users: **500M**
- Daily syncing users (DAU): **50M**
- Peak hour share: ~25% of DAU sync within a 2-hour commute window in any given timezone
- Typical sync batch: ~5–15 minutes of buffered samples sent in one HTTP POST

Sample volume per device

- Heart rate: 1 sample per **5 s** = 17,280 samples/day
- Steps: 1 sample per **1 s** when moving (~12 active hours) $\approx$ 43,200 samples/day
- GPS: only during workouts; $\sim 1 \text{ Hz} \times \sim 45 \text{ min/day}$ for active users $\approx$ 2,700 fixes/day
- Other (sleep stage, calories, SpO2): $\sim 5,000$ samples/day
- Total raw samples per active user: **$\sim 70\text{K/day}$**

Bytes and aggregate ingest

- Raw sample tuple uncompressed: $\sim 24 \text{ B}$ (timestamp + value + tags)
- Daily uncompressed: $70\text{K} \times 24 \text{ B} \approx$ **1.7 MB/user/day**
- With time-series compression (delta-of-delta + Gorilla XOR) $\sim 8\times \rightarrow$ **$\sim 210 \text{ KB/user/day}$**
- Add workout media + metadata blobs $\rightarrow$ **$\sim 14 \text{ MB/user/day}$** total written incl. workout payloads
- Aggregate steady: $50\text{M} \times 14 \text{ MB} \div 86,400 \text{ s} \approx$ **8 GB/sec average**
- Peak commute hour multiplier $\sim 3\times \rightarrow$ **$\sim 25 \text{ GB/sec aggregate ingest peak}$**

Storage tiering

- **Hot 30 days online:** $50\text{M} \times 14 \text{ MB} \times 30 \approx 21 \text{ PB raw} \rightarrow \sim 2.6 \text{ PB compressed in TSDB}$
- **Warm 1 year compressed:** $\sim 31 \text{ PB}$ on cheaper tier (HDD-backed columnar)
- **Cold lifetime:** S3 / Glacier; tens of PB per year of corpus growth

KEY

Key Numbers

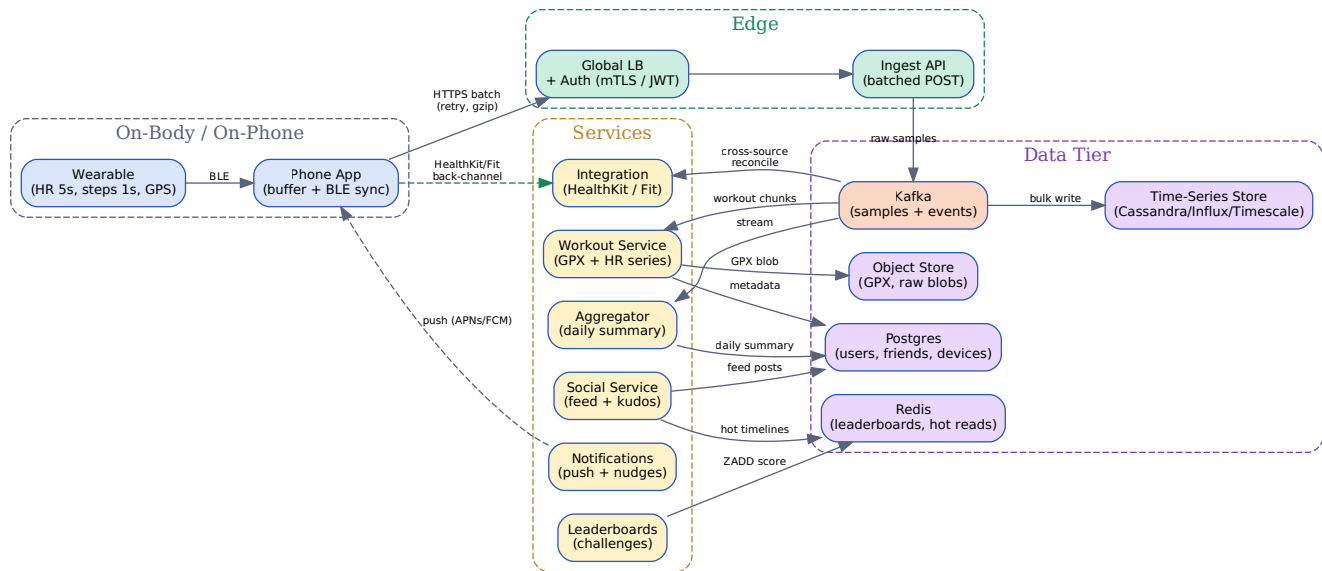
500M users · 50M DAU · $\sim 14 \text{ MB/user/day}$ · **$\sim 25 \text{ GB/s peak ingest}$** · $\sim 2.6 \text{ PB hot}$ · 30d hot / 1y warm / lifetime cold.

04

High-Level Architecture

End-to-end system overview

The pipeline starts at a wearable that streams over BLE to the user's phone. The phone app is the **first durable buffer** — it collects samples for minutes to hours and POSTs batches to the Ingest API when connectivity allows. From there, samples fan out to the time-series store, the aggregator, the workout service, and the social and integration services.



End-to-end fitness platform: device → phone → batched ingest → fan-out into TSDB, aggregator, workouts, social, integrations.

Component responsibilities

- **Phone app:** the canonical buffer; decodes BLE, deduplicates, batches, retries with exponential backoff
- **Ingest API:** stateless, validates schema and auth, writes raw to Kafka before ack — never blocks on TSDB
- **Kafka:** the durable spine; partitioned by user_id so a single consumer sees a user's stream in order
- **Time-series store:** Cassandra or Influx or TimescaleDB — keeps 30d hot online
- **Aggregator:** stream job that materialises per-day summaries on the fly
- **Workout service:** upgrades a stream chunk into a structured workout document with simplified GPX
- **Social service:** friend graph, fan-out, auto-generated activity posts, privacy filter
- **Integration service:** HealthKit / Google Fit pairing, conflict resolution across sources
- **Leaderboards:** Redis sorted sets, expire weekly/monthly

05

Sync Model — Device → Phone → Cloud

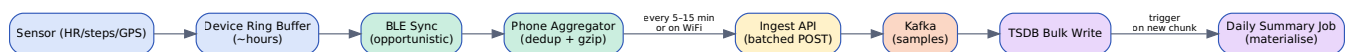
Bursty, offline-tolerant batching

Three-hop journey

1. **Sensor** → **wearable RAM**: on-watch ring buffer of last few hours
2. **Wearable** → **phone (BLE)**: opportunistic, when the phone is in range
3. **Phone** → **cloud (HTTPS batch)**: when WiFi/cellular is healthy
4. Each hop is best-effort with idempotent retries; only the cloud write is the durable commit

Why batched, not streaming

- **Battery**: a radio on for 1 second of bulk transfer beats 60s of trickling
- **Connectivity**: users spend hours offline (gym, subway, flights) — a streaming socket would just churn
- **Cost**: each request has fixed overhead — 5–15 minute batches cut request count by orders of magnitude
- **Compression**: larger payloads compress better (~8× on time-series)



Sample-batch ingest path: device buffer → phone aggregator → bulk-write to TSDB → trigger daily summary job.

Idempotency and dedup

- Each sample carries a stable (`device_id`, `timestamp`, `metric`) key — re-sending is a no-op
- Phone tags each batch with a `batch_uuid` the cloud uses to short-circuit replays
- Server stores `last_synced_at` per device — phone can ask “what's my high-water mark?” after reinstall

Backfill on reconnect

- Phone offline for days → device buffer wraps; oldest samples dropped at the device layer
- If phone is online but cloud is unreachable, phone keeps batches in SQLite up to ~1 GB
- On reconnect, phone uploads in chronological order, throttled to avoid swamping a freshly-recovered ingest tier
- Server marks each backfilled day for **re-aggregation**: the daily summary is recomputed

06

Time-Series Storage Choice

Cassandra vs Influx vs Timescale for samples

The time-series store is the load-bearing component: ~25 GB/s peak ingest, billions of points per day, predictable per-user range scans, and aggressive retention tiering. The three credible candidates are **Cassandra with TTL**, **InfluxDB**, and **TimescaleDB**.

Comparison

Aspect	Cassandra (with TTL)	InfluxDB	TimescaleDB
Write model	LSM, partition by (user, day)	TSM, time-bucketed	Postgres + hypertable chunks
Throughput	Excellent at very high write QPS	Excellent for time-series writes	Good; limited by Postgres write path
Compression	Good with chunk-aligned tables (~5×)	Best-in-class for samples (~10×)	Native columnar compression on chunks (~8×)
Ad-hoc query	Limited; pre-design partitions	Flux/InfluxQL; flexible time-window	Full SQL + joins to user metadata
Retention TTL	Per-row TTL native	Retention policy per bucket	Drop old chunks (cheap)
Operational fit	Battle-tested at PB scale (Strava-class)	Purpose-built TSDB	Familiar SQL ops
Cost shape	Cheap on commodity SSD	Cheap; vendor lock-in risk	Mid; pays for SQL flexibility

Recommendation

TIP

Pick Cassandra for samples; Postgres/Timescale for metadata

Use **Cassandra (with per-row TTL)** for the raw sample stream — predictable partition design (`user_id`, `day_bucket`), excellent write throughput, and TTL is free retention. Use **Postgres or TimescaleDB** for user metadata, daily summaries, friend graph, and workout records — anything queried by humans or joined to user data.

Cassandra schema for samples

```
-- partition: one row per (user, day, metric); clustering by timestamp
CREATE TABLE samples (
  user_id      bigint,
  day_bucket   date,
  metric       text,          -- 'hr', 'steps', 'gps_lat', 'gps_lon', ...
  ts           timestamp,
  value        double,
  source       text,          -- 'watch', 'phone', 'healthkit', 'fit'
  PRIMARY KEY ((user_id, day_bucket, metric), ts)
) WITH CLUSTERING ORDER BY (ts ASC)
AND default_time_to_live = 2592000 -- 30 days hot
AND compression = {'class': 'LZ4Compressor'};

-- workouts go in their own table for blob-style access
CREATE TABLE workouts (
  user_id      bigint,
  workout_id   timeuuid,
  started_at   timestamp,
  ended_at     timestamp,
  sport        text,
  summary      frozen<map<text,double>>, -- distance, calories, ...
  gpx_blob_url text,                    -- pointer to S3
  hr_series_url text,                    -- compressed series in S3
  privacy      text,                    -- 'private'|'friends'|'public'
  PRIMARY KEY (user_id, workout_id)
) WITH CLUSTERING ORDER BY (workout_id DESC);
```

Per-sample row vs blob-per-day

Strategy	Pros	Cons
Per-sample row	Easy ad-hoc query, simple ingest path	Row overhead; bulkier on disk; index pressure
Blob-per-day per metric	~10× smaller on disk; trivial range read	Hard to update mid-day; batch must replace blob
Hybrid (chunk per hour)	Compromise; small re-write window	Two code paths; chunk boundary edge cases

INSIGHT

Hybrid in practice

Most production fitness backends store per-sample rows in Cassandra for the hot 30 days, then a nightly compactor rewrites finalised days into compressed columnar blobs in object storage for the warm tier. The query layer routes by age.

07

Aggregations & Daily Summary

From raw samples to insights

Per-day rollout at ingest

- Stream consumer reads from Kafka samples topic, partitioned by user
- Maintains an **in-memory per-user accumulator** keyed by (user_id, day)
- On a tumbling 1-minute window, flushes incremental updates to the daily-summary row in Postgres
- Tracked metrics: total steps, active minutes, calories, distance, avg HR, max HR, resting HR proxy
- Late-arriving samples (backfill) bump a dirty flag → recompute that day end-to-end

Sliding-window aggregation

```
# Stream consumer pseudo-code (Flink/Kafka Streams style)
def on_sample(user_id, day, metric, ts, value):
    state = state_store.get((user_id, day)) or empty_summary()
    if metric == 'steps':
        state.steps += value
        if value > 0:
            state.active_seconds.add(ts // 60) # bitmap of active minutes
    elif metric == 'hr':
        state.hr_sum += value
        state.hr_count += 1
        state.hr_max = max(state.hr_max, value)
    elif metric == 'gps':
        state.distance_m += haversine(state.last_gps, value)
        state.last_gps = value
    state_store.put((user_id, day), state)

# Every 60s flush dirty users to Postgres summary table
for (user_id, day), state in dirty_iter():
    pg.upsert('daily_summary', user_id, day, state.to_row())
```

Nightly weekly/monthly rollout

- Off-peak (per timezone) cron pulls finalised daily rows from Postgres
- Computes weekly_summary and monthly_summary; writes back to Postgres
- Computes **training-load** and **personal records** deltas — feeds insights
- Job is idempotent and cursor-based — restarting reprocesses only its range

daily_summary table

```
CREATE TABLE daily_summary (  
  user_id          BIGINT,  
  day              DATE,  
  steps            INT,  
  active_minutes   INT,  
  distance_m       INT,  
  calories         INT,  
  hr_avg           SMALLINT,  
  hr_max           SMALLINT,  
  resting_hr       SMALLINT,  
  sleep_minutes    INT,  
  sources          TEXT[],    -- which providers contributed  
  dirty            BOOLEAN DEFAULT FALSE,  
  updated_at       TIMESTAMPTZ DEFAULT NOW(),  
  PRIMARY KEY (user_id, day)  
);  
CREATE INDEX idx_dirty_days ON daily_summary(day) WHERE dirty;
```

08

Workouts — GPX Tracks & HR Series

First-class structured records

Why workouts are different

- Workouts have a **begin/end boundary** the user explicitly demarcated
- They carry GPS — large per-event payload (a 1-hour run is ~3,600 fixes ≈ 50 KB compressed)
- They are **shared on the feed** — the social product hangs off the workout
- They get displayed on a map — geometry must render fast even on slow phones

Storage layout

- **Metadata** (sport, duration, distance, splits, sport-specific stats): Postgres row
- **Raw GPX**: stored as compressed blob in S3, addressed by workout_id
- **Simplified track** (for map display): generated server-side, cached in S3 too
- **HR series**: dense per-second array, stored as blob; hot reads hit Redis

GPS track simplification (Ramer-Douglas-Peucker)

```
# Reduce a GPX track to map-display-friendly geometry.
# Recursively keep the point with max perpendicular distance to the
# segment from start to end; drop anything within epsilon.
def rdp(points, epsilon):
    if len(points) < 3:
        return points
    dmax, idx = 0.0, 0
    for i in range(1, len(points) - 1):
        d = perp_distance(points[i], points[0], points[-1])
        if d > dmax:
            dmax, idx = d, i
    if dmax > epsilon:
        left = rdp(points[:idx + 1], epsilon)
        right = rdp(points[idx:], epsilon)
        return left[:-1] + right
    return [points[0], points[-1]]

# epsilon ~5 m on city runs, ~25 m on bike rides — tuned by sport.
# 10 km run: ~3,600 fixes → ~150 polyline points (24× smaller).
```

Full GPX vs simplified track

Use	Track	Why
Map preview in feed	Simplified	Tiny payload, renders instantly, hides exact route
Detailed analysis screen	Full	User wants pace/elevation per segment
Export (.gpx file)	Full	Other tools expect raw fidelity
Public sharing	Simplified + privacy zone clip	Reduces inadvertent home-address leak

workouts metadata table

```
CREATE TABLE workouts (  
  workout_id      UUID PRIMARY KEY,  
  user_id         BIGINT NOT NULL,  
  sport           TEXT,           -- 'run','bike','swim','strength'  
  started_at      TIMESTAMPTZ,  
  ended_at        TIMESTAMPTZ,  
  distance_m      INT,  
  duration_s      INT,  
  elevation_gain  INT,  
  avg_hr          SMALLINT,  
  calories        INT,  
  privacy         TEXT,           -- 'private'|'friends'|'public'  
  gpx_full_url    TEXT,           -- s3://...  
  gpx_simple_url  TEXT,           -- s3://...  
  hr_series_url   TEXT,           -- s3://...  
  created_at      TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_workouts_user_time ON workouts(user_id, started_at DESC);  
CREATE INDEX idx_workouts_public   ON workouts(started_at DESC) WHERE privacy='public';
```

09

Real-Time Leaderboards & Challenges

Redis sorted sets, weekly/monthly windows

Mechanics

- Each challenge has a key like `lb:steps:weekly:2026W18`
- Member is `user_id`, score is the cumulative metric for the window
- Aggregator **increments the score** as samples flow in: `ZINCRBY lb:steps:weekly:W18 1234 user_id`
- Top-N queries: `ZREVRANGE lb 0 99 WITHSCORES` — $O(\log N + M)$
- Member rank: `ZREVRANK` — let the user see their own position even if not top-100
- Set `EXPIRE` to ~1 week past window end to auto-clean

Real-time leaderboard vs nightly batch

Approach	Pros	Cons
Real-time (Redis ZINCRBY)	Live experience; gamification works	Memory cost; abuse-vector if not deduped
Nightly batch	Cheap; simple	Stale; users can't see today's progress
Hybrid: realtime today + batch finalize	Best UX; bounded cost	Two code paths; reconcile on day-end

Cheating and abuse

- Cap implausible per-minute deltas (e.g. > 250 steps/min sustained)
- Reject samples whose `source` is unverified for prize-tier challenges
- Server-side tally is the source of truth; client-reported totals never feed leaderboards

TIP

Sharded sorted sets

At 50M DAU, a single ZSET is too large. Shard the leaderboard (e.g. by `user_id % 64`) into 64 sub-leaderboards, then merge top-N at read with a heap. Sacrifices exact global rank for users outside top-N, but keeps each ZSET in healthy memory range.

10

Social Feed & Activity Posts

Friend graph, fan-out, kudos, comments

What is unique here

- **Posts are auto-generated** from completed workouts and milestones — users rarely author them directly
- Auto-summary posts: 'Joe ran 10 km at 5:12/km' or 'Aisha hit a 7-day step streak'
- Engagement: kudos (like), comments, share — same patterns as Instagram
- Privacy-by-default per activity (private / friends / public)

Feed model

- Friend graph in Postgres (`follows(follower_id, followee_id)`)
- **Fan-out on write** for normal users: when a workout finalises, push to friends' timeline lists in Redis
- **Fan-out on read** for celebrities (Strava-famous athletes with millions of followers)
- Hybrid threshold: if followee has > 50K followers, fall back to read-time pull
- Privacy filter applied at both write (skip private) and read (re-check, in case privacy changed)

Auto-post trigger

```
# When a workout is finalised:
def on_workout_finalised(w):
    if w.privacy == 'private':
        return
    post = build_auto_post(w)          # pre-rendered card
    feed_db.insert_post(post)
    if w.privacy in ('friends', 'public'):
        followers = graph.followers(w.user_id, privacy_min=w.privacy)
        if len(followers) < 50_000:
            for f in followers:
                redis.lpush(f'timeline:{f}', post.id)
                redis.ltrim(f'timeline:{f}', 0, 999)
        else:
            mark_pull_only(post)        # serve via fan-out on read
```

Kudos and comments

- Kudos: counter on the post + per-user dedup (a user can kudo each post once)
- Comments: append-only table keyed by `post_id`; small, paginate by id

- Notifications: Kafka → Notifications service → APNs/FCM push to phone

11

HealthKit & Google Fit Integration

Two-way sync and conflict resolution

Two-way sync model

- On iOS, the phone app reads HealthKit and writes back via the HealthKit API; on Android, the same with Google Fit
- **Inbound**: HealthKit/Fit samples are batched alongside watch samples — same Ingest API, different source tag
- **Outbound**: when our backend records a workout, it surfaces it in HealthKit/Fit so other apps see it
- All sync goes through the user's phone — there is no direct cloud-to-cloud channel with Apple

The duplicate problem

- A 7,000-step walk recorded by an Apple Watch can arrive from **three sources**: the watch directly, the phone's motion coprocessor, and HealthKit aggregating both
- Naively summing them triple-counts the walk
- Dedup must be source-aware and time-aware

Conflict-resolution rules

Conflict	Rule
Watch HR + phone HR overlap	Prefer watch (closer to body, more accurate)
Steps from watch + phone	Prefer watch when worn; fall back to phone when not
HealthKit reports an aggregate that overlaps a known source	Drop the aggregate, keep the source-tagged samples
Two devices recorded same workout	Keep both, but mark one as authoritative for the feed
Manual entry vs sensor	Sensor wins; manual flagged as 'self-reported'

Mechanism

- Every sample carries source and device_id
- The Integration service runs a **reconciliation pass** per (user, day) when multiple sources have written for that day

- Reconciliation produces a canonical `view_samples` projection — that is what summary jobs read
- Raw samples are never deleted — the canonical view is rebuilt deterministically from priority rules

WARNING

Don't trust client-side dedup

HealthKit's own dedup is per-app and unreliable across devices — never assume the phone has already deduplicated. Always keep raw samples server-side and compute the canonical view yourself.

12

Privacy — GPS Zones & Sharing Controls

Sensitive data, sensible defaults

Why this is a first-class concern

- GPS tracks reveal home, work, school, places of worship, sensitive routines
- A public Strava heatmap once accidentally exposed military base layouts — privacy is not theoretical
- Users grant location with the expectation we do not weaponise it back to them

Per-activity privacy

- **private**: never appears in feed, never aggregated into public leaderboards
- **friends**: visible only to mutual or one-way followers
- **public**: visible to anyone; still privacy-zone clipped
- Default is configurable per user — the safer default for new accounts is *friends*

Privacy zones

- User defines circles (**center**, **radius 200–1000 m**) around home, work, etc.
- When a track is rendered for any non-self viewer, points inside any zone are **clipped before serving**
- Distance and time totals are preserved; only the geometry is hidden — the user's stats are unchanged
- Server-side enforcement: clipping happens in the workout service, not the client

```
def clip_track_for_viewer(track, owner_zones):
    out = []
    skipping = False
    for p in track:
        if any(haversine(p, z.center) < z.radius for z in owner_zones):
            skipping = True
            continue
        if skipping:
            out.append(GAP_MARKER)    # leaves a visible gap on map
            skipping = False
        out.append(p)
    return out
```

HIPAA-adjacent vs HIPAA

- By default this is **not** HIPAA — fitness data from consumer devices is not PHI on its own

- If we partner with a healthcare provider (clinical trial, employer wellness, EHR write-back), the integrated subset becomes PHI
- We isolate that subset behind a separate VPC, BAA-bound vendors, and stricter audit logging

WARNING

Encrypt GPS at rest, audit access

GPS samples are encrypted at rest with a per-user data key (envelope encryption via KMS). All internal access to GPS data is logged to an append-only audit table; analytics queries that touch raw lat/lon must go through a privileged service account.

13

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Phone offline for days	Backlog of samples on device	Server-side stale last-sync alert	Phone backfills in chronological order; aggregator marks days dirty and recomputes
Clock skew on device	Out-of-order or future-dated samples	Compare timestamps to server clock at ingest	Snap to server time on ingest if drift > 60s; preserve original in raw_ts
Duplicate samples (multi-source)	Triple-counted steps	Reconciliation diff vs canonical view	Source-priority dedup; rebuild canonical view per day
Kafka backlog	Aggregations lag	Consumer lag metric	Auto-scale stream consumers; back-pressure on ingest only after Kafka itself is at risk
Cassandra hot partition	Latency spike for one user	Per-partition latency monitor	Day-bucket partitioning already caps; shard further by metric if a user is a torture-test case
S3 GPX write fails	Workout missing route	Workout finalisation retry counter	Retry with backoff; mark workout 'route pending' so the feed still shows summary
HealthKit sends garbage	Implausible aggregates	Sanity bounds on totals	Reject samples failing bounds; flag user for support if persistent
Region failure	Users in region cannot sync	Synthetic probe failures	Phone retries with exponential backoff; route to next region; ingest is offline-tolerant

Recovery posture

- **RPO ≈ 0** for samples once cloud has acked — Kafka replicates 3×
- **RPO ≈ minutes** for derived state (summaries, leaderboards) — they can be rebuilt from raw
- **RTO < 30 min** for ingest path; the rest of the system can lag
- Phone is the disaster shock absorber: it can buffer ~1 GB locally, so a 6-hour cloud outage is invisible to the user

WARNING

Never lose a sample after ack

The phone treats a 200 from Ingest as a durable commit and frees its buffer. Therefore Ingest must write to Kafka (3× replicated) *before* returning 200 — never just an in-memory queue. Losing samples after ack is the worst-class bug for a fitness platform: users see their PRs evaporate.

14

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Sample granularity	HR 5s, steps 1s, GPS only in workouts	Higher fidelity costs storage and battery; chosen rates match what users perceive.
Sync model	Batched HTTP from phone	Up to 15 min ingest lag; vastly cheaper and battery-friendlier than streaming.
TSDB choice	Cassandra (with per-row TTL)	Operationally heavier than Influx but predictable at PB scale.
Aggregation timing	Real-time daily + nightly weekly/monthly	Live numbers users see today; cheap batch for longer windows.
Feed fan-out	Hybrid: write for normals, read for celebrities	Two code paths, but caps memory blowup for high-fan-out users.
GPS sharing	Simplified track + privacy zone clip	Tiny payload, safer; loses some fidelity for public viewers.
Leaderboards	Real-time Redis ZSET (sharded)	Memory cost; necessary for the gamification UX.
Conflict resolution	Server-side reconciliation per (user, day)	Extra compute; the only reliable path given multi-source duplicates.
Privacy default	<i>friends</i> for new accounts	Slightly less viral; meaningfully safer.
Hot retention	30 days online	Most product needs are within 30 days; older data warm/cold is fine.

15

Interview Playbook

How to present this in 45 minutes

This system is broad — keep the narrative on a leash. The interviewer wants to see you handle bursty time-series ingest, retention tiers, multi-source dedup, and privacy. The social layer and integrations are supporting acts.

45-minute outline

1. **Intro (3 min):** 500M users / 50M DAU / ~14 MB/user/day / ~25 GB/s peak ingest
2. **Sync model (5 min):** device → phone → cloud, batched, offline-tolerant — explain why not streaming
3. **High-level arch (5 min):** Ingest → Kafka → TSDB / Aggregator / Workouts / Social / Integrations
4. **Time-series store (8 min):** Cassandra vs Influx vs Timescale; partition by (user, day, metric); per-row TTL
5. **Aggregations (5 min):** live daily summary + nightly weekly/monthly; dirty-day recomputation
6. **Workouts & GPX (5 min):** structured doc, RDP simplification, full vs simple track
7. **Social + leaderboards (5 min):** hybrid fan-out, sharded Redis ZSET
8. **HealthKit/Fit + privacy (6 min):** source-priority dedup, privacy zones, HIPAA boundary
9. **Failure modes (3 min):** phone offline backfill, clock skew, duplicate samples

Numbers to memorise

- ~25 GB/s peak aggregate ingest
- ~14 MB/user/day after compression incl. workouts
- 30 days hot · 1 year warm · lifetime cold
- HR 5s · steps 1s · GPS only in workouts
- Fan-out threshold ~50K followers (write-fan-out below, read-fan-out above)

Common follow-ups

- **Q: Why not a streaming WebSocket?** A: Battery and connectivity. Devices are offline often; batched HTTP with retries is cheaper, more reliable, and gets the same effective freshness once you accept a 5-15 minute lag.
- **Q: How do you handle a 5-day backlog after a vacation?** A: Phone buffers locally, uploads chronologically with throttling on reconnect; aggregator marks each affected day dirty and recomputes the daily summary.
- **Q: How do you stop someone gaming step counts?** A: Server-side tally only, plausibility caps per minute, source-tagged samples, drop unverified sources for prize-tier challenges.

- **Q: A user lives next to a gym they hate — how do they hide it?** A: Privacy zone — a circle around the address. Server clips the track segment inside that circle for any non-self viewer; the user's totals stay correct.
- **Q: Strava-famous athlete with 3M followers posts a workout — how does it scale?** A: Cross the fan-out threshold, switch to read-time pull. Their post sits in a per-user posts list; followers' feed builders fetch and merge at read time.
- **Q: What if Apple HealthKit and our watch disagree on step count?** A: Source-priority rules — watch wins for HR and steps when worn. Reconciliation runs per (user, day) and rebuilds a canonical view; raw samples are never deleted.
- **Q: Is this HIPAA?** A: Not by default. The moment we integrate with a healthcare provider for clinical data exchange, the integrated subset becomes PHI and lives in an isolated, BAA-bound environment.

KEY

Headline Numbers

500M users · 50M DAU · ~14 MB/user/day · ~25 GB/s peak ingest · 30d hot / 1y warm / lifetime cold · HR 5s / steps 1s / GPS in workouts only · Fan-out flips at ~50K followers.